

Using LLMs for Use Case Modelling of IoT Systems: An Experience Report

Mirza Rehenuma Tabassum
Toronto Metropolitan University
Toronto, ON, Canada
mirza.tabassum@torontomu.ca

Sadaf Mustafiz
Toronto Metropolitan University
Toronto, ON, Canada
sadaf.mustafiz@torontomu.ca

Matthew J. Ritchie
Toronto Metropolitan University
Toronto, ON, Canada
m1ritchie@torontomu.ca

Jörg Kienzle
ITIS Software, University of Málaga
Málaga, Spain
joerg.kienzle@uma.es

ABSTRACT

Requirements engineering (RE) plays an essential role in the success of system and software development. Textual use case models are valuable for capturing diverse scenarios describing the interactions between the system and its actors, but their development, particularly for the Internet of Things (IoT), can be tedious and error-prone due to the added complexities and heterogeneous nature of such systems. Automating requirements elicitation and specification tasks with the use of generative AI is highly desirable. This paper explores the potential of large language models (LLMs) for generating interaction models for IoT systems from informal problem descriptions. We investigate the capabilities of OpenAI's GPT-4 and Google's Gemini for generating standard and UCM4IoT textual use cases by carrying out a comparative study using four IoT applications. While both of these LLMs show promise as supporting tools, our findings indicate a need for further refinement and domain-specific training to enhance their precision and reliability in requirements development for the IoT domain.

CCS CONCEPTS

• **Software and its engineering** → *Requirements analysis*.

KEYWORDS

Requirements Engineering, Use Case Modelling, Large Language Model, LLM, Model-Based Development

ACM Reference Format:

Mirza Rehenuma Tabassum, Matthew J. Ritchie, Sadaf Mustafiz, and Jörg Kienzle. 2024. Using LLMs for Use Case Modelling of IoT Systems: An Experience Report. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24)*, September 22–27, 2024, Linz, Austria. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3652620.3687810>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MODELS Companion '24, September 22–27, 2024, Linz, Austria

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0622-6/24/09
<https://doi.org/10.1145/3652620.3687810>

1 INTRODUCTION

The Internet of Things (IoT) represents a heterogeneous network of interconnected objects, machines, software, and people. The *things* that constitute an IoT system must interact with each other to achieve some common goal. Therefore, the many facets of the IoT – hardware, software, network, and environment – must be taken into consideration when specifying system interactions [6].

Efficient and effective requirements development plays a vital role in the engineering of IoT systems [24]. The heterogeneous nature of such systems makes requirements development for the IoT domain even more complex. Among the languages and methods available, textual use case models are used as a fundamental technique in this context. These models provide a structured narrative that encapsulates various scenarios, interactions, and functionalities, offering a comprehensive framework for understanding and expressing the needs of stakeholders [15]. However, the process of gathering and refining requirements expressed with use cases can be lengthy and resource-intensive. Incomplete sets of requirements and error-prone models that contain inaccuracies or inconsistencies can significantly slow down system development and lead to project delays and cost overruns.

To face these challenges, we explore the capability of state-of-the-art Large Language Models (LLM), specifically GPT-4 [2] and Gemini [1], to generate precise and complete textual use cases for IoT systems. The study seeks to assess the LLMs' potential as an alternative to conventional approaches for scenario-based requirements development. GPT-4 and Gemini were selected for this experiment due to their widespread recognition and adoption as leading LLMs. Their extensive pre-training on diverse datasets and sophisticated architectures make them well-suited for natural language processing tasks, including textual use case generation.

Research Questions To derive meaningful insights from our experiment, a series of research questions have been formulated. The questions are designed to comprehensively explore the efficacy and precision of LLM-generated use case models for IoT systems with standard and extended use case modelling techniques. We have used the Fondue [21] template for standard and UCM4IoT [8] for extended use case modelling.

RQ1. How syntactically correct are LLM-generated textual use case models?

RQ2. How semantically correct are LLM-generated use case models?

RQ3. Are LLMs capable of identifying exceptions and generating corresponding handlers when developing UCM4IoT use cases?

RQ4. Are different LLMs equally suitable for generating use case models?

The rest of this paper is organized as follows: Section 2 provides background and discusses related work on LLM-based requirements engineering. Section 3 describes the process followed to generate use cases. Section 4 presents the quantitative and qualitative results. Section 5 discusses the experiment with Gemini. Section 6 elaborates on some limitations of this work, and section 7 concludes the paper.

2 BACKGROUND & RELATED WORK

2.1 Use Case Modelling

2.1.1 Textual Use Cases. Use case modelling, recognized as a fundamental technique for the elicitation and specification of system requirements, employs textual descriptions to articulate scenarios or behaviours that satisfy user goals [11, 15]. Use cases encompass a main success scenario delineating the path to achieving a goal, along with an extensions section detailing alternate scenarios. External entities that interact with the system are called *actors*, and can be primary (pursuing a goal with the system), or secondary (from whom the system requires assistance). Unlike UML use case diagrams, which offer an overview of system goals and associated actors, textual use cases cover detailed interactions between the system and the actors in the environment.

There exist in the literature several templates for modelling textual use cases. In this work, we used the well-defined Fondue use case model template [21]. In Fondue, each use case template specifies a name, scope, use case level, and use case multiplicity. Then follows a numerated sequence of *interaction steps* that comprise the main success scenario, followed by an extension section that specifies alternative scenarios, if any. Each step in a use case should focus on a single input or output interaction between the system and one or several actors. Furthermore, complex interactions can be decomposed into a use case hierarchy, where a step in a higher-level use case can invoke a lower-level use case. Samples of Fondue textual use cases and the corresponding use case diagram are available in [21]. In this paper, we refer to this traditional use case modelling language as the *standard* one.

In *Exceptional Use Cases* [22], standard use cases have been extended with concepts from the exception handling world, i.e., exceptions and handlers. Based on exceptional use cases, DREP [17] defines a requirements elicitation approach for dependable reactive systems, that systematically guides the developer to consider reliability and safety concerns. With specific questions, the developer is lead to discover exceptional situations that change the context in which the system operates or threaten to fail user goals, and document how the system should handle these situations.

While the above works have extended standard use case approaches, other approaches have formalized standard use cases further. For example, the *Restricted Use Case Model* (RUCM) defines a metamodel that enforces the content of a step in a use case template to either represent an action (for interactions) or a keyword (for control flow directives) [23]. An action can be one of five possible interactions, either between the system and an actor (inputs

from a primary actor, or outputs to either a primary or a secondary actor), or internally to the system (in the case of validation or modification of the internal state). A keyword step typically represents control flow directives, or a reference to another use case. RUCM also places certain restrictions on the language that can be used in step definitions, e.g., that steps must be written in an active voice.

Similarly, Concern-oriented Use Cases (CoUC) [16] is a use case modelling language designed to support planned and opportunistic reuse. CoUC also provides a metamodel that defines the hierarchical structure and behavioural scenario descriptions for use cases.

2.1.2 UCM4IoT. UCM4IoT is a use case-based modelling language designed to support requirements development for the Internet of Things [8], [7]. The Internet of Things (IoT) refers to a network of interconnected physical devices, vehicles, appliances, and other objects embedded with electronics, software, and network connectivity that enable these objects to connect, collect and exchange data [6]. The main components include sensors for data collection, actuators for controlling the physical environment, and communication networks for data transmission.

Just like RUCM and CoUC, UCM4IoT provides a comprehensive use case template and imposes a rigorous structure on the text used to describe the steps of the scenarios. Additionally, UCM4IoT addresses the unique heterogeneity and adaptive nature of IoT systems by offering domain-specific language constructs. This includes modelling different facets of IoT systems and incorporating exceptional situations and adaptive behaviours. UCM4IoT extends traditional textual use cases with IoT-specific elements, such as sensors, actuators, tags, and readers, and supports effective discovery and specification of exceptional scenarios and handling mechanisms. Exceptions encompass atypical scenarios disrupting standard interaction sequences and necessitating distinct handling procedures. Four types of exceptions are defined: software, hardware, network and environment. To address these exceptions, specialized use cases called *handlers* are developed. Figure 1 illustrates a UCM4IoT textual use case model. This use case is developed to fulfil a primary actor's, *Customer* (human-type), goal *Shopping*. It invokes three lower-level use cases, and identifies three potential environment exceptions in the extension block.

2.2 Related Work on LLM-based Requirements Engineering

The potential of LLMs in aiding the requirements development process is demonstrated in [10], [9], and [19]. These studies showcased ChatGPT's ability to understand and generate different requirements models based on given prompts, offering a new avenue for AI-driven requirements engineering. However, as Cámara et al. [9] point out, while ChatGPT excels in creating syntactically correct models, it faces challenges in semantic accuracy and response consistency in complex tasks. In their work, they investigated the capabilities of ChatGPT in automating the generation of UML class diagrams. They focused on evaluating how well ChatGPT could create syntactically and semantically accurate UML diagrams and its effectiveness in understanding software design tasks.

Arulmohan et al. [5] explore the utilization of LLMs, specifically ChatGPT in extracting concepts of domain models from user stories, particularly from agile backlogs. They conducted an analysis

```

use case: Shopping
scope: SmartStoreSystem
level: USER_GOAL
intention: "The customer intends to shop for items in
the smart store."
multiplicity: "Multiple customers can shop at the same
time."
primary actor: HUMAN::Customer::1..*
main success scenario:
1. [EnterStore] "The customer enters the store"
"Step 2 can be repeated according to the intent of
the Customer."
2. [AddToCart] "The customer adds an item to the cart."
3. [ExitStore] "The customer exits the store."
use case ends in: SUCCESS
extensions:
exception for (1-3):
(1-3)a. {ENVIRONMENT_EXCEPTION::FireHazard} #Fire
hazard in the store. Customer needs to leave the store
immediately, and no new customer can enter.#
use case ends in: FAILURE
exception for (1-3):
(1-3)b. {ENVIRONMENT_EXCEPTION::CriminalAttack} #Store is
under attack. No new customer can enter.#
use case ends in: FAILURE
alternative for 2:
2a. [ReturnItem] "Customer returns an item."
use case continues at step: 3
alternative for 2:
"Customer does not add anything to the cart."
use case continues at step: 3

```

Figure 1: UCM4IoT: a Sample Use Case Model

comparing the effectiveness of ChatGPT against CRF, a traditional NLP technique, and the VN tool. Their findings indicated that while ChatGPT showed promise in rapidly generating domain models, it was outperformed by CRF in terms of accuracy and consistency.

Ronanki et al. [19] present a comprehensive analysis of ChatGPT's assistance in the requirements elicitation process, particularly for developing trustworthy AI systems. The findings show that ChatGPT's generated requirements scored highly in terms of understandability, consistency, and correctness, yet they lagged in clarity and feasibility. This evaluation highlights the potential and limitations of using LLMs like ChatGPT in RE, especially in tasks that require a high degree of precision and feasibility.

De Vito et al. [14] developed ECHO, a method using ChatGPT to enhance the quality of UML use cases. ECHO combines co-prompt engineering with iterative feedback by using ChatGPT for model completion to improve the quality of use case descriptions. Their controlled experiments with software professionals showed ECHO's superiority in improving use case clarity, correctness, and completeness.

Ruan et al. [20] explore the use of ChatGPT in requirements modelling for embedded systems with a framework for automatic model generation. Specific prompts for ChatGPT are designed to extract essential elements from informal requirements texts and structured format, which are then assembled into a model using predefined rules. A case study on a digital home system demonstrates the framework's efficacy in expediting the modelling process, albeit with some limitations in completeness and accuracy.

Abualhaija et al. [4] investigated ChatGPT's use for generating UML sequence diagrams from requirements. Their study evaluated the quality of generated diagrams based on criteria such as completeness, correctness, and adherence to standards. While the results

showed promise in understandability and standard compliance, issues were identified in completeness and correctness, particularly when dealing with ambiguous or inconsistent requirements.

Conrardy and Cabot [12] examine the potential of LLMs like GPT-4 and Gemini to convert hand-drawn UML class diagrams into formal models. Their findings indicate that while LLMs can facilitate model-driven engineering, the accuracy and completeness of these conversions vary with the model and input complexity, necessitating human oversight. This contrasts with our focus on textual data representations.

The reviewed literature collectively indicates that while GPT shows promise in automating RE, their effectiveness varies. Studies highlight their capability to create structurally correct models but also point out limitations in handling complex and nuanced requirements. Although the potential exists for LLMs in requirements engineering, there is currently a lack of published work on directly utilizing such techniques for generating textual use case models. While the assessment of ChatGPT for UML class diagram generation has been conducted [9], use case descriptions differ significantly from UML diagrams in that they focus on interaction modelling and are textual models. Therefore, we conducted a comprehensive evaluation of use case descriptions generated by LLMs to determine their syntactic and semantic accuracy.

3 EXPERIMENT SETUP

3.1 Research Methodology

We designed a *benchmarking study* that assesses the suitability of LLMs by applying them to generate standard and extended use case models of four representative examples from the IoT domain. We assess the results quantitatively through a set of metrics, and qualitatively based on a detailed human examination of the generated use cases.

3.2 Benchmark Examples

To the best of our knowledge, there is currently no openly available benchmark for use case-based requirements engineering. The repository described by Dalpiaz [13] contains user stories that are limited in scope, brief, informal, and often overlook comprehensive scenarios and alternative considerations. We, therefore, assembled our own set of evaluation IoT examples. A summary of the informal requirements of the examples is presented below. The complete descriptions are available at <https://github.com/mde-tmu/LLM4UCM/tree/main/ProblemDescription> and could serve as a benchmark for future experiments on use case-based requirements elicitation.

- The **407 ETR** system is an automated toll collection for an expressway that runs north of Toronto, Canada. It dynamically calculates tolls based on vehicle size and type upon entry and exit, sending monthly bills to vehicle owners. The 407 ETR has been used on several occasions in teaching standard use case-based requirements elicitation using Fondue, as well as discovering exceptional situations using DREP [18].
- **BIXI** is a public bicycle-sharing system in Montréal with a network of bike docks and pay stations. It offers utility bicycles with a unisex step-through frame and three gears, which can be rented using a subscription or access code obtained at a pay station. The

system is monitored in real-time and fees are structured to incentivize shorter trips. The BIXI system has been used in teaching standard use case-based requirements elicitation using Fondue, with a special focus on modularity.

- The **Smart Store** provides manual checkout-free shopping based on automated item detection and customer recognition. Customers register with a mobile application or a pre-registered credit card, allowing the system to identify items (with cameras and sensors) and link them to virtual carts. Automatic payment occurs upon exiting. Staff are differentiated from customers to avoid billing errors. This example has been used for teaching the use of UCM4IoT.
- The **Smart Fire Alarm** system, integrating smoke, heat, and carbon dioxide sensors, provides rapid notification to homeowners and fire departments via mobile application alerts, triggers sprinklers for preemptive action, and allows manual control for false alarms. The Smart Fire Alarm has been used for teaching the use of UCM4IoT.

The above examples were selected to encompass a diverse range of IoT applications, representing varying levels of complexity and scale. The smart store illustrates consumer-oriented applications, while the smart fire alarm system exemplifies safety-critical use cases. Additionally, the 407 ETR system and BIXI showcase a traffic infrastructure-based IoT application. This selection allows for a comprehensive evaluation of our approach across different domains and complexity levels within the IoT ecosystem.

Furthermore, Fondue and UCM4IoT-based requirements had previously been elaborated for all four IoT systems by humans, allowing us to compare the use cases generated by the LLMs with a "ground truth".

3.3 Comparison Metrics

We defined a set of metrics to quantitatively evaluate the performance of the LLMs. For each of the benchmark examples, we have a "ground truth" (*GT*), a solution that was established by humans without the use of LLMs and any form of automated assistance. These were developed in an academic setting by software engineering faculty and graduate students with a high level of expertise in use case modelling. We then compare the generated use cases with the "ground truth" use cases to determine whether a) the LLM discovered the elements that are in *GT* (#overlap), b) the LLM did not discover the elements that are in *GT* (#missing), c) the LLM discovered new model elements that are relevant for the system, but were not in *GT* (#new). Finally, we also report on d) whether the LLM has generated model elements that are not relevant to the system (#excess). The judgment of whether model elements correspond (for determining #overlap, #missing, and #new), and whether elements are relevant for the system (for determining #excess) was made by human inspection that was performed by the authors.

We manually inspect the generated use cases and count:

- *The number of goals*: The number of goals in the human-generated use cases is a high-level quantification of the functionality of the system. Hence an interesting number to look at is the **overlap** between the LLM-generated use cases and the human-developed ones. It is also interesting to look at the **new** numbers, for they represent cases in which the LLM discovered useful new functionality that the human did not consider. Finally, the number of

excess goals quantifies how much useless functionality the LLMs produce.

- *The number of use cases*, excluding handler use cases: The number of use cases is a measurement of the decomposition granularity. Use case based approaches usually recommend that use cases should focus on one goal or subgoal, and have between 3-12 steps, i.e., not be too big, because that makes them complicated to read, and not too small because then the functionality is split over many use cases.
- The number of primary actors
- The number of secondary actors
- The number of total steps including interaction steps, internal processing, control steps, steps in handler use cases, and alternate steps
- The number of correctly formed interaction steps
- The number of exceptions in UCM4IoT use cases
- The number of handler use cases in UCM4IoT use cases

3.4 Constructing the Prompts

Table 1: Prompt for Generating Fondue Use Cases.

Prompt (<i>Fondue</i>)
Assume you are a software engineer developing requirements for an IoT system.
Generate detailed standard textual use case models following the Fondue template. Identify all the goals of the system and write a use case for each goal. A goal can be defined at different levels of abstraction: summary, user goal, and sub-functional.
To help you understand the process, I am attaching a lecture note on developing requirements with use cases.
Please read the attached document. For every use case, there will be one primary actor, and there can be one or multiple secondary actors. Except, a subfunction level use case, where there may not be a primary actor.
An interaction step can either refer to a lower-level use case, or describe a base interaction step between the system and the environment (where the system must interact with at least one actor), or describe an optional system processing step or communication happening in the environment.
I am providing you with examples of standard use cases. Please generate use case models after reading the problem statement.
«Add multiple examples»
«Add the problem statement»

GPT-4 was directed to assume the role of a software requirements developer tasked with generating use case models from the given problem statements. We used OpenAI's Playground [3] with the GPT-4o model and kept the temperature and top P to 0.5 for optimal output. Table 1 and table 2 present the prompts used for standard and UCM4IoT use case generation purposes.

GPT-4, not originally trained with Fondue use case templates and UCM4IoT models, required supplementary training for these specific techniques. To this end, GPT was provided with lecture notes on requirements development with standard use case models and a research article on UCM4IoT [7]. Despite GPT-4's capability to process these documents, there were indications of its inability to parse or integrate some critical details fully. Hence, sample use

Table 2: Prompt for Generating UCM4IoT Use Cases.

Prompt (UCM4IoT)
Assume you are a software engineer developing requirements for an IoT system. Generate detailed UCM4IoT use-case models. Identify all the goals of the system and write a use case for each goal. A goal can be defined at different levels of abstraction: summary, user goal and sub-functional. To help you understand the process, I am attaching a research article on developing requirements with use cases. Please read the attached document. For every use case, there will be one primary actor, and there can be one or multiple secondary actors. Except in a subfunction-level use case, where there may not be a primary actor. There are different types of steps in these use-case models. An invocation step refers to a lower-level use case, an interaction step describes a base interaction step between the system and the environment (where the system must interact with at least one actor), and an internal processing step describes optional system processing steps or communication happening in the environment. Please identify any exceptions that may exist in any of these interaction steps. Exceptions are categorized in four types: Software, Hardware, Network, and Environment. Also, please write handler use cases for these exceptions. I am providing you with examples of standard use cases. Please generate use-case models after reading the problem statement. «Add multiple examples» «Add the problem statement»

case models for standard and UCM4IoT were also provided. To guide LLMs to either produce standard or UCM4IoT use cases, the prompts were tailored to match each template.

4 EXPERIMENT RESULTS FOR GPT

4.1 Quantitative and Qualitative Results

The outputs generated by GPT were manually checked by the authors for compliance with the requested use case syntax, and then the metrics presented in Section 3.3 were evaluated. Table 3 shows the quantitative results of our experiment for each of the 4 examples for both Standard and UCM4IoT use cases.

- **Goals:** The number of goals in GPT-generated Standard use cases compared to human-developed ones (#overlap) are 4/5 for the Smart Store, 2/2 for the Smart Fire Alarm, 1/6 for the 407 ETR, and 3/5 for Bixi. For UCM4IoT use cases, it is 3/5, 1/2, 2/6, and 2/5 respectively. GPT also discovered some new, useful functionality. In the Smart Store Standard use cases, GPT discovered an additional functionality related to *configuring the payment system*.
- **Use Cases:** The number of use cases in the GPT-generated use cases compared to the human-generated ones is 67.6% in Standard use cases and 43% in UCM4IoT.
- **Primary Actors:** GPT found 2 primary actors compared to the human-developed 4 for the smart store, 1/4 for the smart fire alarm, 1/1 for the 407 ETR, and 2/1 for the Bixi when generating UCM4IoT use cases. It had 1 excess primary actor (*PHYSICAL_ENTITY::BikeDock*) for Bixi. For the Standard use cases, it found 2/4 primary actors for the smart store, 1/1 for the smart fire alarm, 3/1 for 407 ETR, and 1/1 for Bixi. It found 2 excess actors (*Registered Vehicle*, *Unregistered Vehicle*) for the 407 ETR system.

```

72 scope:      SmartStoreSystem
73 level:      USER_GOAL
74 intention:  "The customer intends to add an item to their virtual cart."
75 multiplicity: "Many customers can add items to their carts at the same time."
76 primary actor: HUMAN::Customer::1..*
77 secondary actor: SENSOR::ShelfSensor::1..*
78 main success scenario:
79     1. "The Customer picks up an item from the shelf."
80     2. "The ShelfSensor detects the removal of the item."
81     3. "The system adds the item to the Customer's virtual cart."
82 use case ends in: SUCCESS
83 extensions:
84     alternative for 2:
85         2a. "The ShelfSensor fails to detect the removal of the item."
86         2a.1. "The system uses the EntryCamera to verify the item removal."
87 use case continues at step: 3
88     exception for 3:
89         3a. {HARDWARE_EXCEPTION::SensorFailure} "Shelf sensor failure."
90 use case ends in: FAILURE

```

Figure 2: GPT-generated Model: UCM4IoT Use Case.

- **Secondary Actors:** GPT generated UCM4IoT use cases have 5/10 secondary actors for the smart store system, 3/9 for the smart fire alarm system, 4/12 for the 407 ETR, and 2/5 secondary actors for the Bixi application. In particular, for the 407 ETR system, it added 1 new actor and 1 excess actor (*TAG::Transponder*, *SOFTWARE::BillingSystem*). For the Standard use cases, it found all 5 secondary actors for the smart store system, 7/9 for the smart fire alarm system, 6/10 for the 407 ETR, and 5/5 in the Bixi application. Among those, GPT generated 2 new (*OnlineSystem*, *Transponder*) and 1 excess (*BillingSystem*) actors for 407 ETR, and 2 new (*MobileApp*, *OnlineSystem*) for the Bixi system.
- **Steps:** GPT generated a total of 41 steps in UCM4IoT use cases compared to human-developed 146 steps for the smart store, 39/42 for the smart fire alarm, 27/90 for the 407 ETR, and 24/83 for the Bixi. In the case of Standard use cases, GPT generated 36/126, 26/51, 62/63, and 46/76 steps respectively. We also checked how many interaction steps GPT generated correctly and the numbers are 11/103, 9/15, 9/57, and 9/48 in all four applications respectively. For Standard use cases, there are 6/41, 11/35, 8/40, and 23/48 interaction steps compared to the human-developed steps.
- **Exceptions:** For the exceptions (#overlap), GPT discovered 5/11 for the smart store, 5/18 for the smart fire alarm, 2/12 for the 407 ETR, and 0/3 for Bixi. However, GPT discovered 1 new exception for the 407 ETR (*NetworkUnreachable*) and the smart store (*RegistrationError*), 2 new for the smart fire alarm (*SettingsSaveError*, *FireDepartmentUnreachable*), and 2 new for Bixi (*NetworkUnreachable*, *SubscriptionVerificationFailed*).
- **Handlers:** GPT generated 2 handlers (#overlap) out of 8 handlers for the smart store, 4/13 for the smart fire alarm, 0/11 for the 407 ETR, and 0/1 for Bixi. The only handler GPT generated for 407 ETR and Bixi is a new handler, i.e. *ReconnectNetwork*. In the smart fire alarm, it also generated a new handler (*HandleFireDepartmentUnreachable*).

4.2 Interpretation

In this subsection, we interpret the results to answer our research questions, RQ1-RQ3.

RQ1. GPT correctly followed the syntax of use case models in most cases. However, we have noticed in some of the use case descriptions, that it did not include step numbers in the main success scenario, though it did not miss the numbering for the alternate scenarios.

Table 3: GPT Results for Standard and UCM4IoT Use Cases

Metrics	Smart Store			Smart Fire Alarm			407 ETR			BIXI		
	Human	Standard	UCM4IoT	Human	Standard	UCM4IoT	Human	Standard	UCM4IoT	Human	Standard	UCM4IoT
User Goals	5	overlap: 4 missing: 1 new: 2	overlap: 3 missing: 2	2	overlap: 2 excess: 2	overlap: 1 new: 1 excess: 1	6	overlap: 1 new: 2 excess: 1	overlap: 2 new: 1 missing: 4	5	overlap: 3 new: 2 missing: 2	overlap: 2 missing: 3
Primary Actors	4	overlap: 2 missing: 2	overlap: 2 missing: 2	Standard: 1 UCM4IoT: 4	overlap: 1 missing: 0	overlap: 1 missing: 3	1	overlap: 1 excess: 2	overlap: 1 excess: 0	1	overlap: 1 excess: 0	overlap: 1 excess: 1
Secondary Actors	Standard: 5 UCM4IoT: 10	overlap: 5 excess: 0	overlap: 5 missing: 5	9	overlap: 7 missing: 2	overlap: 3 missing: 6	Standard: 10 UCM4IoT: 12	overlap: 3 new: 2 excess: 1	overlap: 2 new: 1 excess: 1	5	overlap: 3 new: 2 missing: 2	overlap: 2 missing: 3
Steps	Standard: 126 UCM4IoT: 146	36	41	Standard: 51 UCM4IoT: 42	26	39	Standard: 63 UCM4IoT: 90	62	27	Standard: 76 UCM4IoT: 83	44	24
Exceptions	11	-	overlap: 5 new: 1 missing: 6	18	-	overlap: 5 new: 2 excess: 1	12	-	overlap: 2 new: 1 excess: 1	3	-	overlap: 0 new: 2 missing: 3
Handlers	8	-	overlap: 2 missing: 6	13	-	overlap: 4 new: 1 missing: 9	11	-	overlap: 0 new: 1 missing: 11	1	-	overlap: 0 new: 1

RQ2 and RQ3. To answer RQ2 and RQ3 we inspect each section of the use case templates individually.

- **Goals:** GPT-4 demonstrated acceptable performance in identifying significant goals from problem descriptions, yet it exhibited limitations in capturing the full spectrum of necessary use cases. For example, in the smart store system, it failed to generate the user goal of returning an item, a crucial aspect of the shopping experience. On the other hand, it generated all goals for the Smart Fire Alarm. It had more trouble with the UCM4IoT use cases.
- **Use Cases:** The result shows that GPT consistently generated use cases with a lower level of detail. We noticed inconsistencies in GPT's understanding of use case abstraction levels. In some cases, GPT incorrectly identified the level, e.g. it labelled *Enter Store* in the smart store system as a summary level use case without invoking any lower-level use cases.
- **Primary Actors:** In 3 of the 4 examples, GPT discovered all the primary actors of the Standard use cases. In the smart store system, it did not find *Admin* and *ServicePerson* as primary actors in both UCM4IoT and Standard use cases, although the problem description touched on these aspects. Additionally, it failed to identify three primary actors in the UCM4IoT use cases for the smart fire alarm system. This probably happened as GPT predominantly categorized *Human* type actors as primary actors across most use cases. We noticed, that GPT correctly discovered *Human* type primary actors in UCM4IoT, but could not find any *Software* or *Device* types of actor. Although it added a *Physical_Entity* actor in Bixi, that was an excess primary actor.
- **Secondary Actors:** GPT failed to discover many secondary actors in all four applications. The main and alternate steps indicate that it often skipped a lot of details in the system-actor interactions, which may have been the result of missing secondary actors. It also showed a tendency to misclassify some actors in the UCM4IoT use cases. For example, *Gantry* in 407 ETR system should be a *Physical_Entity* but GPT classified it as a sensor. Despite missing some secondary actors, GPT also discovered a few new actors. It added *Transponder* in 407 ETR, *MobileApp* in Bixi, and an *Online_System* in both of these applications.
- **Steps:** GPT-4 generated a significantly lower number of steps in the main success scenario and alternate steps block compared to the human-developed use case descriptions. It frequently omitted granular, atomic steps in use case descriptions, opting instead to

transition to subsequent stages without comprehensive elaboration. This tendency to bypass detailed step-by-step explanations may hinder a thorough understanding of the specific actions and interactions involved in each use case. Furthermore, these steps lack actor-system interaction that defines interaction steps. This undermines the comprehensibility and value of the generated steps, as they fail to capture the essential system-actor interactions.

RQ3. Exceptions. GPT did not perform as per expectation in discovering exceptions for UCM4IoT use cases. It only discovered 36% exceptions in four applications. Moreover, it did not discover any environment exception in any of these applications. We also noticed, GPT mostly added similar network exceptions. It also generated some software and hardware exceptions. In examining the capabilities of GPT, it appears that it discovered generic exceptions mostly, such as *NetworkUnreachable*, *SensorFailure*. Figure 2 shows a UCM4IoT use case generated by GPT-4.

Handlers. Writing handlers for every exception is required in UCM4IoT. Nonetheless, GPT only generated 39% handlers compared to human-developed use cases. The quality of the handlers is also not satisfactory. Its capability to generate handlers was limited to retrying to connect to the network or replace the faulty hardware. For example, for network failure and unreachable exceptions, the handler only included one or two steps in the system's attempt to reconnect to the network in different applications' use cases. Interestingly, although the Standard use case does not require handlers, GPT added a use case named *Handle No Internet Connection* to provide functionality to the user who is not connected to the network while checking out from the smart store.

4.3 Key Findings and Recommendations

The results of this experiment highlight both the potential and limitations of GPT-4 in the use case modelling of IoT systems. The results revealed GPT's proficiency in structuring use cases and identifying primary actors, but highlighted significant deficiencies in discovering secondary actors and interactions essential for IoT systems. GPT successfully identified many IoT-specific actors, however, it sometimes misclassified actors within UCM4IoT use cases. Additionally, it demonstrated a shortfall in recognizing several secondary actors, indicating its limited capability to extract detailed information from problem descriptions.

The experiment underscored GPT's limitations in generating use cases for complex, self-adaptive, heterogeneous environments, as evidenced by its inconsistent performance across different systems and its failure to capture nuanced system interactions comprehensively. GPT's proficiency in identifying exceptions and composing handlers for UCM4IoT use cases remains inadequate. Most of the exceptions it discovered were network and hardware types. The exceptions it discovered tend to be overly simplistic and generic, while the handlers it generates are rudimentary and lack details. It also failed to write handlers for all the discovered exceptions.

The lessons learned from this experiment are multifaceted. Firstly, while GPT can serve as valuable tools for initial use case generation, its outputs require substantial review and refinement by human experts, particularly in contexts where contextual understanding is critical. Secondly, the experiment underscores the necessity for enhanced training, possibly incorporating more domain-specific knowledge and examples that can better prepare the GPT model for the complexities of IoT systems.

In conclusion, GPT's utility as a tool in the RE process for critical systems like the IoT remains limited. Its current application should be approached with caution, requiring significant enhancements in domain-specific training and model understanding to improve reliability and accuracy in use case generation.

Future research should focus on methods to better equip LLMs with the capability to handle the complexities associated with IoT systems and improve the accuracy and completeness of generated use case models. Additionally, exploring the integration of LLMs with existing requirements engineering tools and methodologies could further enhance their practical applicability in real-world software development projects.

5 COMPARISON OF GEMINI AND GPT

While the initial experiments provided insights into research questions RQ1, RQ2, and RQ3, we needed to run additional experiments to address RQ4. We conducted the same experiment, i.e., generating standard and UCM4IoT use cases using Google Gemini. This section presents an analysis of the use case generation capabilities of Gemini and then compares it with GPT-generated use cases. It focuses on the metrics used for the overall experiment. We investigated how Gemini performs compared to human-developed and GPT-4 use cases. This work with Gemini is exploratory and intended to investigate differences between LLMs.

On Google AI Studio, both standard and UCM4IoT use case generation used Gemini's 1.5 Flash Model with *temperature* set to 0 and *top_P* left at its default (0.95). The complete set of use cases generated by Gemini can be found at <https://github.com/mde-tmu/LLM4UCM/tree/main/Application>.

5.1 Experiment with Gemini: Findings

Compared to the human-generated use cases, Gemini-generated use cases require significant syntactic corrections to adhere to the correct use case syntax. Figure 3 shows a sample UCM4IoT summary-level use case generated by Gemini with syntactical issues. Gemini-generated primary actors of type "VEHICLE" and some secondary actors of type "GANTRY". This labelling does not adhere to the UCM4IoT syntax as the only defined actor types are

human, software, physical entity, or device (sensor, actuator, tag, reader). Many of the main success scenarios and alternate steps are also syntactically incorrect since they do not include at least an actor (primary or secondary) and the *system's* interaction within the step, or numbered steps.

```
3=Use Case: ProcessRegisteredVehicle
4 Scope: ETRSystem407
5 Level: SUMMARY
6 Intention: A registered vehicle owner intends to use the 407 ETR highway.
7=Multiplicity: Many registered vehicles can use the highway simultaneously.
8 Primary Actor: VEHICLE::RegisteredVehicle::1..*
9 Secondary Actor:
10 GANTRY::EntryGantry::1..1
11 GANTRY::ExitGantry::1..1
12 SOFTWARE::AccountManagementSystem::1..1
13 Main Success Scenario:
14="The RegisteredVehicle enters the highway and passes under the EntryGantry."
15 [IdentifyTransponder] "The EntryGantry identifies the transponder on the RegisteredVehicle."
16 [RecordEntryData] "The EntryGantry records the entry time, date, and location."
17 "The RegisteredVehicle exits the highway and passes under the ExitGantry."
18 [IdentifyTransponder] "The ExitGantry identifies the transponder on the RegisteredVehicle."
19 [RecordExitData] "The ExitGantry records the exit time, date, and location."
20 [CalculateToll] "The AccountManagementSystem calculates the toll based on distance and time of day."
21 [DebitAccount] "The AccountManagementSystem debits the associated account."
22 "The ExitGantry provides a green signal and four beeps to the RegisteredVehicle."
23 Use Case Ends in: SUCCESS
```

Figure 3: Gemini UCM4IoT Use Case for the 407 ETR.

Gemini-generated summary-level use cases encompassed several sub-functional level use cases, where the latter's main success scenarios typically consisted of only one or two steps without extensions. Gemini aimed to modularize system functions by defining summary-level use cases as sequences of invocation steps, and sub-functional cases as short, self-contained modules, following common abstraction methodologies in use case generation. It should be noted that Gemini did not define a single user-goal level use case. Additionally, Gemini did not use the level categories (summary, user-goal, sub-functional) appropriately.

Semantically, Gemini-generated some interesting use cases, exceptions, handlers, and actors that the human-written ones did not include. For example, it introduced a subscription management actor in the BIXI use cases, a system management interface actor in the Smart Store, and both exceptions and handlers for data storage failures in the 407 ETR system. These additions enriched the use cases with more detailed parts for secondary actors and exceptions. For example, in the BIXI system's UCM4IoT use cases, Gemini developed 16 use cases and 10 handlers. Notably, none of the exceptions generated by Gemini overlapped with the human-identified handlers. New exceptions, such as software exceptions *DataAnalysisFailure*, *VirtualCartUpdateFailure*, and *FireDepartmentSystemFailure*, were generated by Gemini as it modelled data analysis capabilities within the suggested use cases. While exceptions like the above would be considered in the design phase as opposed to the requirements engineering phase, these new additions are worth mentioning, because the initial problem statement did not mention analysis capabilities. By introducing new use cases, actors, exceptions, and handlers that could enhance the system, Gemini demonstrates its potential as a requirements generation tool and also as a gap analysis tool.

Table 4: GPT vs Gemini Results for Standard and UCM4IoT Use Case Generation

<i>Standard</i>	BIXI			407ETR			Smart Store			Smart Fire		
	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>
Use Cases	8	7	14	10	7	15	20	8	13	11	8	8
Primary Actors	2	1	3	1	3	3	4	5	1	1	4	1
Secondary Actors	5	5	8	10	6	7	5	4	12	10	4	7
Total Steps	76	44	97	63	62	104	126	36	84	51	26	53
Interaction Steps	48	23	43	40	8	36	41	6	20	35	11	19
Token Count	N/A	4171	9331	N/A	5585	10496	N/A	4360	8666	N/A	3543	6326
<i>UCM4IoT</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>	<i>Human</i>	<i>GPT</i>	<i>Gemini</i>
Use Cases	8	5	16	10	5	15	21	7	17	27	7	8
Primary Actors	2	2	2	1	3	2	4	2	2	4	1	1
Secondary Actors	5	1	6	12	2	9	10	3	9	2	3	3
Total Steps	83	24	69	90	27	104	164	41	69	42	39	40
Interaction Steps	48	9	29	57	9	23	103	11	13	15	9	14
Exceptions	3	2	11	12	1	16	11	4	13	18	6	7
Handlers	1	1	10	11	1	17	8	2	17	13	3	10
Token Count	N/A	4304	17806	N/A	5096	24976	N/A	6447	23505	N/A	5740	12518

5.2 Gemini vs. GPT

Gemini-generated use cases were notably more verbose than those generated by GPT-4. Empirically, it is evident that the use cases generated by Gemini were consistently more detailed than GPT-4. There were only three metrics across all four comparative cases where GPT-4 generated more than Gemini. In Table 4, the token counts for all the Gemini responses are between 9331 and 24976 tokens while GPT's responses were between 4171 and 5585 tokens. The size difference between the two responses indicates a difference in reasoning, or perhaps, the processing power allotted to each model. GPT-4 seemed to focus solely on the system's core functionality, which can be seen through the limited use cases generated by GPT-4 (6.75 generated use cases on average) compared to the human-developed ones. On the other hand, Gemini (13.25 generated use cases on average) generated use cases that included secondary system functions in all four evaluated examples. Whether through built-in system limitations or how these LLMs try to understand the problem, Gemini provided responses between 1.79 and 4.90 times longer than the GPT-4 results.

It is clear from both qualitative and quantitative standpoints that Gemini and GPT performed in different ways. GPT provided concise solutions that aligned well with the human-developed use cases at the cost of not generating use cases that address system concerns not explicitly defined in the problem statement. This succinct type of response is beneficial for describing core functionalities without including many extraneous details. GPT's lack of new additions compared to the original use cases cannot be seen as wholly negative because too many considerations could lead to fuzzy descriptions. When considering Gemini's results, it is almost the antithesis of what was observed from GPT. Gemini typically derived many new secondary actors, use cases, exceptions, and handlers compared to the original examples. Gemini's extensive solutions not only covered core system components but also addressed additional concerns, such as rider safety in the BIXI, different alarm errors in the Smart Fire, and license plate recognition in the 407 ETR system. While the use cases generated by Gemini were more varied than GPT's, a recurring issue of verbosity presented itself. Many of

the handlers generated (for instance, *HandleVehicleClassificationFailure* and *HandleOwnerIdentificationFailure*) were semantically identical to each other except for the exception being handled. In practice, these handlers should have been condensed to address several possible exceptions instead of having numerous identical handlers.

In conclusion, the comparative analysis between Google Gemini and GPT-4 highlights distinct strengths and weaknesses in their approaches to use case generation. GPT excels in producing concise, focused documentation that aligns closely with human-generated use cases, making it suitable for applications where clarity and brevity are paramount. Conversely, Gemini's output is more comprehensive and detailed, capturing a wider array of actors and exceptions, albeit at the cost of increased verbosity and potential redundancy. These findings underscore the importance of selecting the appropriate LLM based on the specific requirements of the task at hand, balancing the need for detailed exploration with the necessity for clear, manageable documentation. Future research could further refine these LLMs, enhancing their ability to produce high-quality, practical use cases that meet diverse user needs.

6 THREATS TO VALIDITY

A crucial limitation in employing LLMs for generating use case models is their dependency on user prompts, which significantly influences the output's relevance and accuracy. To mitigate the potential threats to validity, our prompt for both GPT-4 and Gemini was carefully constructed to elicit accurate and comprehensive use cases. The prompt included explicit instructions to identify actors, their goals, and the system interactions involved in addressing the problem description. Additionally, we provide resources, ensuring a holistic representation of the system's use cases.

Furthermore, the comparison of LLM-generated use cases with the ground truth is impacted by the quality of these ground truths. Variations in experiences with developing use cases, domain knowledge, detailing in requirements, and expression capabilities of these individuals directly affect the quality of the human-developed use cases. While we strive to mitigate this threat by introducing use cases developed by individuals with varying levels of expertise,

requirements development is a creative process, and the inherent differences in human perspectives and approaches remain a potential factor influencing the results.

The human factor also poses a potential threat to the validity of both the qualitative and quantitative results, as they were subjectively calculated and analyzed by the authors. The assessment of semantic correctness could be influenced by individual perceptions. To minimize this risk, each author reviewed the results independently, ensuring that one author's judgment was cross-checked and confirmed by another.

We used different temperature and top P value for GPT and Gemini. While this discrepancy in parameter settings may introduce variability in the generated outputs, our primary goal was to explore how each model performs under conditions that highlight their distinct operational characteristics. However, we recognize that this difference could influence the comparability of results and may have skewed the outcomes in specific contexts. To mitigate this, we conducted a qualitative analysis of the outputs, focusing on the coherence, creativity, and relevance of the generated use cases across different applications. While the varying parameters might impact these attributes, the analysis provided insights into each LLM's behaviour in real-world use case generation scenarios.

Additionally, the completeness and accuracy of the use cases generated by LLMs are contingent on their domain knowledge of the system under study. Since the LLMs' training includes a wide but fixed dataset, their proficiency in certain specialized or novel domains, such as IoT, might be limited, affecting the depth and precision of the generated use cases.

7 CONCLUSION

This study has explored the potential of LLMs, specifically GPT-4 and Gemini, in generating detailed textual use case models for IoT systems. Our findings indicate that while LLMs show promise in automating aspects of requirements engineering, their effectiveness is contingent on well-crafted prompts and domain-specific knowledge. Both GPT-4 and Gemini demonstrated complementary capabilities in identifying system goals and generating use case descriptions, but they also exhibited limitations in capturing the full scope of requirements and differentiating between interaction steps and internal processes. Notably, Gemini produced more verbose and comprehensive outputs compared to GPT-4, highlighting the trade-offs between conciseness and detail in LLM-generated use cases. Based on our observations, these LLMs can be best utilized for generating an initial set of textual use cases from problem descriptions. Subsequently, experts should intervene to review, revise, and refine these use cases to ensure their accuracy and completeness.

Our ultimate goal is to assess the power of generative AI for developing requirements models, both use cases and domain models, from informal problem statements as well as user stories. While the focus of this paper was to assess use case generation capabilities, as the next steps we also intend to explore model completion of use cases using these LLMs.

ACKNOWLEDGMENTS

Sadaf Mustafiz and her students' work has been partially supported by the Natural Sciences and Engineering Research Council

of Canada (NSERC) and Toronto Metropolitan University. Jörg Kienzle's contributions were partially supported by Junta de Andalucía under project QUAL21 010UMA.

REFERENCES

- [1] [n. d.]. Google Gemini. <https://gemini.google.com/>. Accessed: 2024-06-12.
- [2] [n. d.]. OpenAI GPT-4. <https://openai.com/index/gpt-4/>. Accessed: 2024-06-25.
- [3] [n. d.]. OpenAI Playground. <https://platform.openai.com/playground/chat?models=gpt-4o>. Accessed: 2024-06-25.
- [4] Sallam Abualhaija, Chetan Arora, and Alessio Ferrari. 2024. Model Generation with LLMs: from Requirements to UML Sequence Diagrams. In *14th International Model-Driven Requirements Engineering (MoDRE) workshop*. IEEE.
- [5] Sathurshan Arulmohan, Marie-Jean Meurs, and Sébastien Mosser. 2023. Extracting Domain Models from Textual Requirements in the Era of Large Language Models. In *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 580–587.
- [6] Martin Bauer, Nicola Bui, Christine Jarda, and Andreas Nettsträter. 2013. The IoT ARM reference manual. *Enabling Things to Talk* 213 (2013).
- [7] Paul Boutot, Mirza Rehenuma Tabassum, Abdul Abedin, and Sadaf Mustafiz. 2024. Requirements development for IoT systems with UCM4IoT. *Journal of Computer Languages (COLA)* 78 (2024), 101251.
- [8] Paul Boutot, Mirza Rehenuma Tabassum, and Sadaf Mustafiz. 2021. UCM4IoT: A Use Case Modelling Environment for IoT Systems. In *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 767–776.
- [9] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. 2023. On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling* (2023), 1–13.
- [10] Juan Pablo Carvallo and Lenin Erazo-Garzón. 2023. On the Use of ChatGPT to Support Requirements Engineering Teaching and Learning Process. In *Proceedings of the 18th Latin American Conference on Learning Technologies (LACLO 2023)*, Santiago Berrezueta (Ed.). Springer Nature Singapore, Singapore, 328–342.
- [11] Alistair Cockburn. 2001. *Writing effective use cases*. Pearson Education India.
- [12] Aaron Conrardy and Jordi Cabot. 2024. From Image to UML: First Results of Image Based UML Diagram Generation Using LLMs. *arXiv preprint arXiv:2404.11376* (2024).
- [13] F Dalpiaz. 2018. Requirements Data Sets (User Stories)(2018), Mendeley Data, v1. DOI: <https://doi.org/10.17632/7zbb8zsd8y1> (2018).
- [14] Gabriele De Vito, Fabio Palomba, Carmine Gravino, Sergio Di Martino, and Filomena Ferrucci. 2023. Echo: An approach to enhance use case quality exploiting large language models. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 53–60.
- [15] Ivar Jacobson. 1992. *Object-oriented Software Engineering: A Use Case Driven Approach*. ACM Press.
- [16] Ryan Languay, Nika Prairie, and Jörg Kienzle. 2023. Concern-Oriented Use Cases. *Journal of Object Technology* 22, 2 (July 2023), 2:1–14. <https://doi.org/10.5381/jot.2023.22.2.a13>
- [17] Sadaf Mustafiz and Jörg Kienzle. 2009. A Requirements Engineering Process for Dependable Reactive Systems. In *Methods, Models and Tools for Fault Tolerance*, M. Butler, C. Jones, A. Romanovsky, and E. Troubitsyna (Eds.). 220 – 250.
- [18] Sadaf Mustafiz and Jörg Kienzle. 2009. DREP: A requirements engineering process for dependable reactive systems. In *Methods, Models and Tools for Fault Tolerance*. Springer, 220–250.
- [19] Krishna Ronanki, Christian Berger, and Jennifer Horkoff. 2023. Investigating ChatGPT's Potential to Assist in Requirements Elicitation Processes. In *Software Engineering and Advanced Applications (SEAA)*. IEEE, 354–361.
- [20] Kun Ruan, Xiaohong Chen, and Zhi Jin. 2023. Requirements Modeling Aided by ChatGPT: An Experience in Embedded Systems. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. 170–177.
- [21] Shane Sendall and Alfred Strohmeier. 2000. From Use Cases to System Operation Specifications. In *UML 2000 — The Unified Modeling Language*. Springer, 1–15.
- [22] Aaron Shui, Sadaf Mustafiz, Jörg Kienzle, and Christophe Dony. 2005. Exceptional use cases. In *Model Driven Engineering Languages and Systems: 8th International Conference, MoDELS 2005*. Springer, 568–583.
- [23] Tao Yue. 2010. Restricted use case modeling approach (User manual). Norway: Simula Research Laboratory (2010), 1–13.
- [24] Didar Zowghi and Chad Coulin. 2005. Requirements elicitation: A survey of techniques, approaches, and tools. *Engineering and managing software requirements* (2005), 19–46.